

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

7 de septiembre de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 3: Comunicación y coordinación

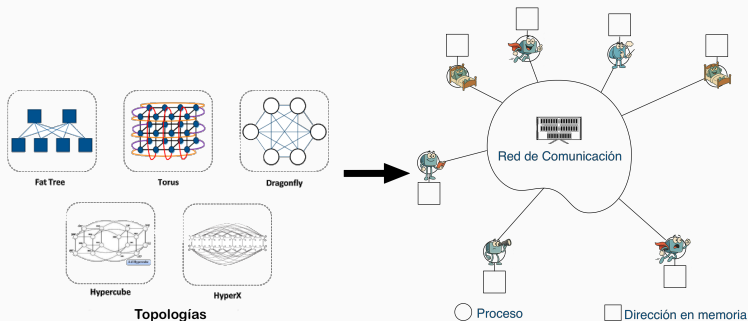
Objetivos:

1. Pasos de Mensaje: MPI, Mensajes Punto a Punto, MPI, Comunicación Colectiva, Blocking vs non-blocking
2. Memoria Compartida: OMP, Constructores y cláusulas.
3. Programacion Híbrida

Message Passing Interface

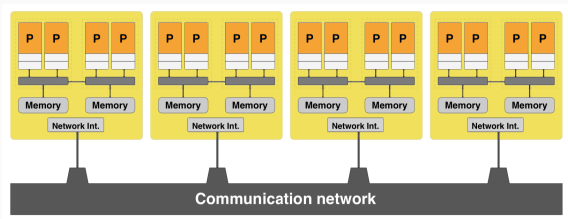
Message Passing Interface

Message Passing denota una forma de comunicación entre procesos, con memoria local, que implica el envío/recibo de mensajes entre ellos.



Message Passing Interface

MPI es un API de comunicación portable de arquitecturas en paralelo.



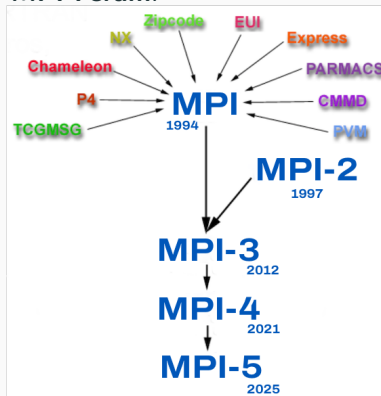
- Utilizado en programación en paralelo y orientada a objetos (C,C++,Fortran)
- Utiliza espacio de memoria particionado en p nodos.
- **MPI** funciona en CPU, Nodo, **NUMA** Non-uniform memory access, cluster. Paralelismo explícito
- **Un proceso MPI** por nodo, o paralelismo híbrido (memoria compartida en cada nodo)

Message Passing Interface

Fue creada en EEUU y Europa, vía el **MPI Forum**.

En junio de 1994 se publica el **standard MPI 1.0**. La interfase MPI permite portabilidad de código (escrito en C, C++, Fortran) a distintas arquitecturas. Utiliza funciones y macros, y **paralelismo explícito**

Ademas, soporta arquitecturas híbridas.



Message Passing Interface

Un proceso MPI
consiste en un código (C++/Fortran),
que participa en un comunicador
y se comunica con otros procesos.

Implementaciones MPI (definen
protocolos de transporte de sdartos):

- **MPICH** (mpich v5.0.1 estable)
- **OPENMPI** (openmpi v5.0.9 estable)

Extensiones (bindings):

Julia, MATLAB, Python, R, Rust

Puede combinarse
con otros métodos de comunicación (e.g.
OpenMP, problemas de sincronización)



OPEN MPI



RUST



julia



MATLAB®

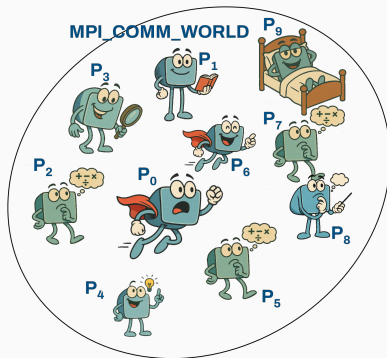


Message Passing Interface: MPI Handles

Son **variables de entorno** en algunas directivas MPI, de tipo entero. Son referencias a estructuras de data en MPI.

MPI_COMM_WORLD: en C, un objeto de tipo MPI_Comm. Representa un comunicador pre-definido conformado por todos los procesos.

MPI_Init() define y llama **MPI_COMM_WORLD** para cada proceso, que define la forma de comunicación entre estos. Este contiene una lista de procesos, que son numerados desde 0, y asignados a la variable **rank**



MPI Errors:

En C, rutinas MPI retornan un entero, con un código de error. De ser detectado, el código aborta

```
1  int ierr;  
2  // . . .  
3  ierr = MPI_Init (&argc, &argv);  
4  // . . .
```

El retorno evaluado como **MPI_SUCCESS** (un entero asociado) representa un correcto funcionamiento de la función MPI.

MPI_Init:

Inicializa MPI. Se llama al inicio y una sola vez en el programa

```
MPI_Init(&argc, &argv)
```

MPI_Comm_size:

Retorna el número total de procesos MPI

```
MPI_Comm_size(comm, &size)
```

MPI_Comm_rank:

Retorna el *rank* del proceso MPI

```
MPI_Comm_rank(comm, &rank)
```

MPI_Wtime:

Retorna el tiempo de reloj en segundos (double precision) en el proceso

```
MPI_Wtime()
```

MPI_Abort:

Termina todos los procesos MPI

```
MPI_Abort(comm, error)
```

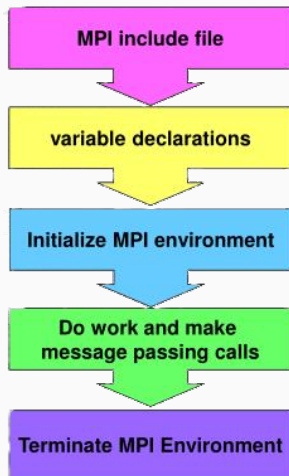
MPI_Finalize:

Termina la ejecución MPI y limpia todas sus estructuras

```
MPI_Finalize()
```

Message Passing Interface

Estructura de un programa en MPI:
Requiere `#include<mpi.h>` Cada proceso está identificado con un número entero (rank), que se cuenta desde cero



Message Passing Interface

Comandos de compilación de programas MPI

Lenguaje	Compilador
C	mpicc
C++	mpiCC, mpicxx, mpic++
Fortran	mpifort, mpif77, mpif90

Compilación en C:

```
mpicc -o ejemplo.exe ejemplo.c
```

Compilación en C++:

```
mpic++ -o ejemplo.exe ejemplo.cpp
```

Ejecución en 4 procesadores:

```
mpirun -np 4 ./ejemplo.exe
```

Ejemplo 01: 'Peekaboo' en MPI

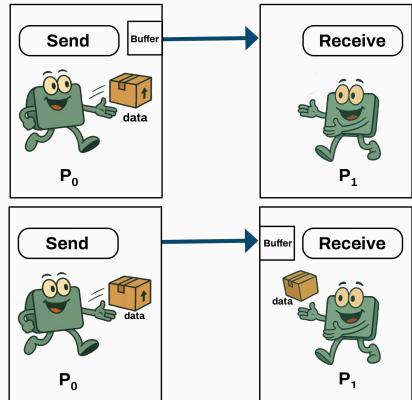
```
1 # include <mpi.h>
2 # include <iostream>
3 using namespace std;
4 int main(int argc, char*argv[])
5 {
6     int rank, size;
7     MPI_Init(&argc,&argv);
8     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9     MPI_Comm_size(MPI_COMM_WORLD, &size);
10    cout<<"Peekaboo! desde "<< rank<<" de "<<size<<endl;
11    MPI_Finalize();
12 }
```

Un mensaje MPI es un array de elementos de un tipo particular MPI enviado, el cual debe ser el mismo que el recibido. MPI soporta así arquitecturas heterogéneas.

Tipo MPI	Tipo C
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

Comunicación punto a punto (bloqueada)

Los tipos de operaciones **punto a punto** ejecutan envío de mensajes entre dos procesadores. Para que la operación de envío y recibo sea segura, la operación se detiene (no retorna) hasta que suceda el envío y recibo.



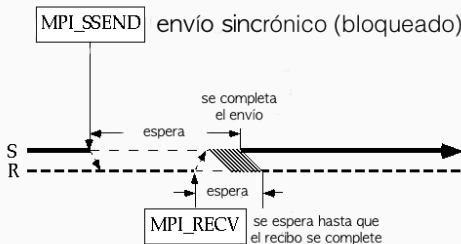
Se clasifican de acuerdo al tipo de envío de mensajes, en:

- **Send sincrónico** , solo se completa cuando ha sido recibido
- **Buffered Send**, siempre se completa (haya sido recibido o no). Se almacena en buffer.
- **Send standard**, sincrónico o buffered
- **Ready Send**, siempre se completa (haya sido recibido o no). No se almacena.
- **Receive**, se completa cuando el mensaje ha sido recibido

Envío y recibo pueden estar sincronizados, durante lo cual se graba la información en memoria (system buffer) reservada para guardar data en tránsito.

Synchronous Send:

Se utiliza si el proceso que envía debe saber si el mensaje ha sido recibido. Para ello el proceso que recibe manda un mensaje de 'recibido' al enviante, momento en el cual se considera el mensaje como enviado



Synchronous Send:

Operación de envío de mensajes.

```
MPI_Ssend(&buffer,count,datatype,dest,tag,comm)
```

- **buffer**: puntero a la variable que ocupa la data que va a ser enviada o recibida (nombre de la variable)
- **count**: número de elementos de la data que serán enviados/recibidos
- **datatype**: tipo de variable MPI
- **dest**: proceso destino del mensaje (rank)
- **tag**: identificador del proceso que envía. Útil en caso de envío de múltiples mensajes
- **comm**: comunicador del grupo de procesos activo

Rutinas MPI de comunicación punto a punto

Buffered Send:

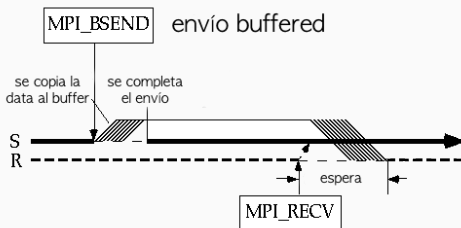
Garantiza un término inmediato a través del almacenamiento en un buffer para transmisión posterior si es necesario.

`MPI_Bsend(&buffer,count,datatype,dest,tag,comm)`

Para ello, el usuario debe destinar suficiente memoria para el programa (en Bytes) utilizando

`MPI_Buffer_attach(buffer,size)`

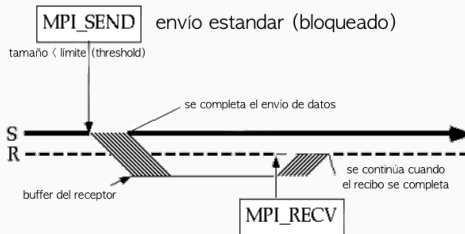
`MPI_Buffer_detach(buffer,size)`



MPI_Send (standard):

Operación de envío de mensajes. Tener en cuenta que:

- No se debe asumir que el envío de mensajes se complete antes de que **receive** empiece
- Puede ser sincrónico o buffered



MPI_Send (standard):

Operación de envío de mensajes.

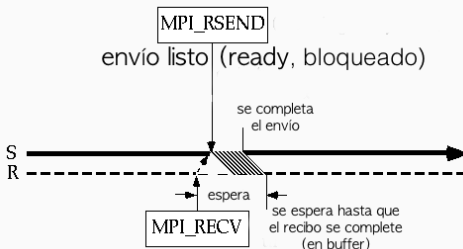
MPI_Send(&buffer,count,datatype,dest,tag,comm)

- **buffer**: puntero a la variable que ocupa la data que va a ser enviada o recibida (dirección de la variable)
- **count**: número de elementos de la data que seran enviados/recibidos
- **datatype**: tipo de variable MPI
- **dest**: proceso destino del mensaje (rank)
- **tag**: identificador del proceso que envía. Útil en caso de envío de múltiples mensajes
- **comm**: comunicador del grupo de procesos activo

Ready Send:

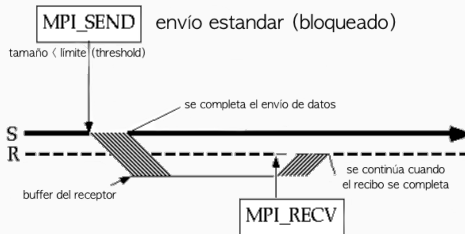
Se completa inmediatamente, enviando el mensaje al comunicador. Este se recibirá solo si el proceso está listo para hacerlo. Si no, ocurrirá un error. El objetivo es mejorar performance. Es un modo difícil para debugging. Solo recomendable si performance es crítica

MPI_Rsend(&buffer,count,datatype,dest,tag,comm)



MPI_Recv (standard blocking):

Operación de recibo de mensajes. Se completan cuando se confirma por parte del proceso destino



MPI_Recv (standard blocking):

```
MPI_Recv(&buffer,count,datatype,source,tag,comm,&status)
```

- **buffer**: puntero a la variable que ocupa la data que va a ser enviada o recibida (dirección de la variable)
- **count**: número de elementos de la data que serán enviados/recibidos
- **datatype**: tipo de variable MPI (el usuario puede crear sus propios tipos de datos)
- **dest**: proceso destino del mensaje (rank)
- **tag**: identificador del mensaje. Útil en caso de envío de múltiples mensajes
- **comm**: comunicador del grupo de procesos activo (MPI_COMM_WORLD)
- **status**: indica el estado del objeto

Se confirma el recibo de mensajes si **source**, **dest**, **tag** corresponden (count y datatype no garantizan la confirmación)

Estado de la comunicación:

Contiene información del remitente, conocida como *status*

status.MPI_SOURCE contiene información de la fuente del mensaje

status.MPI_TAG contiene información del identificador del mensaje

MPI_Get_count(status,datatype,count) contiene el número de elementos recibidos

Ejemplo 02: Send/Recv (1 a 1)

```
1  int main(int argc, char*argv[])
2  {
3      int rank, size;
4      MPI_Init(&argc,&argv);
5      ....
6      if(rank == 0) {
7          // Si esta activo rank 0, number cambia a -1 y se envia al proceso 1
8          number=-1;
9          MPI_Send(&number,1,MPI_INT,1,0,MPI_COMM_WORLD);
10     }
11     else if(rank == 1) {
12         MPI_Recv(&number,1,MPI_INT,0,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
13         ...
14     }
15     MPI_Finalize();
16 }
```

Ejemplo 03: Send/Recv (1 a np)

```
1  int main(int argc, char*argv[])
2  {
3      MPI_Init(&argc,&argv);
4      ....
5      if(rank == 0) {
6          buf=1;
7          for(i=1;i<numprocs;i++)
8              MPI_Send(&buf,1,MPI_INT,i,0,MPI_COMM_WORLD);
9      }
10     else{
11         MPI_Recv(&buf,1,MPI_INT,MPI_ANY_SOURCE,MPI_ANY_TAG,MPI_COMM_WORLD,&stat);
12         ...
13     }
14     MPI_Finalize();
15 }
```

-  <https://www.mpich.org/documentation/guides/>
-  William Gropp, Ewing Lusk and Anthony Skjellum: *Using MPI: Portable Parallel Programming With the Message-Passing Interface*. MIT Press, 3rd edition, Nov. 2014 (336 pages, ISBN 9780262527392)
-  William Gropp, Torsten Hoefler, Rajeev Thakur and Ewing Lusk: *Using Advanced MPI: Modern Features of the Message-Passing Interface*. MIT Press, Nov. 2014 (392 pages, ISBN 9780262527637)
-  Neil MacDonald, Elspeth Minty, Joel Malard, Tim Harding, Simon Brown, Mario Antonioletti: *Parallel Programming with MPI. Historical MPI course notes from EPCC*.
http://www.archer.ac.uk/training/course-material/2014/10/MPI_UCL/Notes/MPP-notes.pdf



Peter S. Pacheco. *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5. Ch 3



Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656. Ch 4

